

蓝皮书 数据结构上机例题

例题 1：单列表排序

编写一个C程序，它能读入若干个整数（以-9999为结束标记，-9999不算在内），并把这些整数按照从小到大次序插入一个单链表内，再把它们以从小到大次序输出在屏幕上，每输出10个换行。

代码清单 1

```
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

/**
 * 单链表
 */
typedef struct node {
    int d; // 值
    struct node * next; // 后继指针
} Tnode;

/**
 * 插入排序
 */
void InsertSort(Tnode **r) {

    if(*r == NULL) {
        // 初始化树
        *r = malloc(sizeof(Tnode));
        (*r)->next = NULL;
    }
    int d; // 值
    Tnode *p, *f;
    do {
        scanf("%d", &d);
        if (d == -9999) {
            break;
        }
        f = *r;
        // 构建新节点
```

```
p = malloc(sizeof(Tnode));
p->d = d;
// 遍历, 找到节点值小于输入值的节点
while (f->next && d > f->next->d) {
    f = f->next;
}
// 插入该节点
p->next = f->next;
f->next = p;
} while (9);
}

/**
 * 遍历输出每一个节点的值
 */
void print(Tnode *r) {
    int c = 0;
    if (r == NULL) {
        puts("Error:no list!");
        return;
    }
    // 指向首节点
    r = r->next;
    puts("The link is as follows:");
    while (r) {
        printf("%-6d", r->d);
        if (++c % 10 == 0) {
            puts("");
        }
        // 指向下一个节点
        r = r->next;
    }
    puts("");
}

int main() {
    Tnode * r = NULL;
    puts("Input integers ending with -9999");
    InsertSort(&r);
    print(r);
    return EXIT_SUCCESS;
}
```

运行结果 1

```

Input integers ending with -9999
203 11 40 2 0 911 -9999
The link is as follows:
0      2      11      40      203      911

```

图 1-1 随机输入排序

例题 2：二叉树遍历

编写一个C程序，它能读入一个二叉树的中序遍历序列和后序遍历序列，并根据此输出二叉树到序遍历序列和叶节点到个数。假定二叉树到节点值为字符、节点数<100，并且任何两个节点值都不同。

代码清单 2

```

#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <string.h>

void exit(int);
#define MAX 100

/**
 * 定义节点
 */
typedef struct node {
    char d; // 节点值
    struct node * lchild; // 左孩子
    struct node * rchild; // 右孩子
} TNode;

/**
 * 构建树
 */
void MkTree(char in[], int is, int ie, char post[], int posts, int poste,
            TNode **r) {
    int i;
    if (is > ie || posts > poste) {
        *r = NULL;
    }

```

```

    } else {
        *r = malloc(sizeof(TNode));
        (*r)->d = post[poste];
        for (i = is; i <= ie; i++) {
            if (post[poste] == in[i]) {
                MkTree(in, is, i - 1, post, posts, posts + i - is - 1,
                    &(*r)->lchild);
                MkTree(in, i + 1, ie, post, posts + i - is, poste - 1,
                    &(*r)->rchild);

                break;
            }
            if (i > ie) {

                puts("Error:input contains an error!\n");
                exit(9);
            }
        }
    }
}

/**
 * 前序遍历
 */
void preorder(TNode *r) {
    if (r) {
        printf("%c", r->d);
        preorder(r->lchild);
        preorder(r->rchild);
    }
}

/**
 * 求叶子数
 */
int leaf(TNode *r) {
    if (r == NULL) {
        return 0;
    } else {
        if (r->lchild == NULL && r->rchild == NULL) {
            return 1;
        } else {
            return leaf(r->lchild) + leaf(r->rchild);
        }
    }
}

```

```
    }  
  }  
}  
  
int main() {  
  
    TNode *r;  
    char post[MAX], in[MAX];  
    puts("Input inorder and postorder:");  
    gets(in);  
    gets(post);  
    MkTree(in, 0, strlen(in) - 1, post, 0, strlen(post) - 1, &r);  
    puts("The preorder is as follows:");  
    preorder(r);  
    printf("\nThere are %d leaves in the tree.\n", leaf(r));  
  
    return EXIT_SUCCESS;  
}
```

运行结果 2

```
Input inorder and postorder:  
A  
A  
The preorder is as follows:  
A  
There are 1 leaves in the tree.
```

图 2-1 中根 A，后根 A 求先根

```
Input inorder and postorder:  
ABC  
ABC  
The preorder is as follows:  
CBA  
There are 1 leaves in the tree.
```

图 2-2 中根 ABC，后根 ABC 求先根

```
Input inorder and postorder:
ABC
CBA
The preorder is as follows:
ABC
There are 1 leaves in the tree.
```

图 2-3 中根 ABC，后根 CBA 求先根

```
Input inorder and postorder:
BCAED
CBEDA
The preorder is as follows:
ABCDE
There are 2 leaves in the tree.
```

图 2-4 中根 BCAED，后根 CBEDA 求先根

```
Input inorder and postorder:
DBEAFCG
DEBFGCA
The preorder is as follows:
ABDECFG
There are 4 leaves in the tree.
```

图 2-5 中根 DBEAFCG，后根 DEBFGCA 求先根

例题 3：拓扑排序

编写一个C程序，它能读入一个有向图的数据，并输出该有向图的拓扑排序序列。

有向图的输入数据形如：

```
n i0 j0 i1 j1 ... ik jk -1 -1
```

n代表节点个数，i0 j0表示输入的节点，同时表明有向边i0到j0，-1 -1表示输入结束，通过邻接矩阵表示存储。

代码清单 3

```
#include <stdio.h>
#include <stdlib.h>

void exit(int);
#define MAX 100

/**
 * 节点
```

```
*/

typedef struct Graph {
    int arcs[MAX][MAX]; // 邻接矩阵
    int vexnum; // 入度最大值
} MGraph;

/**
 * 计算入度值
 */
void Cal_in(MGraph *g, int in[]) {
    int i, j;
    for (i = 0; i < g->vexnum; i++) {
        in[i] = 0;
    }
    for (i = 0; i < g->vexnum; i++) {
        for (j = 0; j < g->vexnum; j++) {
            if (g->arcs[i][j]) {
                in[j]++;
            }
        }
    }
}

/**
 * 拓扑排序
 */
void TopSort(MGraph *g, int in[]) {
    int s = -1, i, j, count = 0;

    for (i = 0; i < g->vexnum; i++) {
        if (in[i] == 0) {
            in[i] = s;
            s = i;
        }
    }

    while (s != -1) {
        printf("V%-3d", s);
        count++;
        i = s;
        s = in[s];
        for (j = 0; j < g->vexnum; j++) {
            if (g->arcs[i][j]) {

```

```

        in[j]--;
        if (in[j] == 0) {
            in[j] = s;
            s = j;
        }
    }
}

if (count < g->vexnum) {
    printf("The graph contains a circle!\n");
}

}

/**
 * 输入
 */
void InputG(MGraph * g) {
    int n, i, j;
    printf("Input the vertex number(in[1.. %d]):\n", MAX);
    scanf("%d", &n);
    if (n < 1 || n > MAX) {
        printf("Error: the vertex number must be in[1.. %d]!\n", MAX);
        exit(-2);
    }
    g->vexnum = n;
    for (i = 0; i < n; i++) {
        g->arcs[i][j] = 0;
    }
    printf("Input the edges i j(i,j must be in[0.. %d]):\n", n - 1);
    do {
        scanf("%d%d", &i, &j);
        if (i == -1 && j == -1) {
            break;
        }
        if (i < 0 || i >= n || j < 0 || j >= n) {
            printf("Error:i and j must be in[0.. %d],Try again!\n", n - 1);
            continue;
        }
        g->arcs[i][j] = 1;
    } while (9);
}

int main(void) {

```

```

MGraph g;
int InDegree[MAX];
InputG(&g);
Cal_in(&g, InDegree);
TopSort(&g, InDegree);

return EXIT_SUCCESS;
}

```

运行结果 3

```

Input the vertex number(in[1.. 100]):
5
Input the edges i j(i,j must be in[0.. 4]):
4 0 4 2 4 3 3 1 0 3 2 0 2 3 2 1 0 1 -1 -1
V4 V2 V0 V3 V1 |

```

图 3-1 求拓扑排序

例题 4：AVL 树

编写一个C程序，它能读入 n ($n \leq 100$) 个两两不相等的整数（输入中两个相邻整数之间有1个或多个空格，最后一个整数是-9999，它是输入结束标记，不作为 n 个整数中的一个），构造一颗包含这 n 个整数（把每一个整数看成一个关键字）的AVL树，再输出该AVL树的前序遍历序列、中序遍历序列和高度。

代码清单 4

```

#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>

#define MAX 100

typedef struct node {
    int d;
    struct node *lchild, *rchild;
} TNode;

void InsertSort(int a[], int *al, int x) {
    int i, low, mid, up;

```

```
low = 0;
up = *a1 - 1;
while (low <= up) {
    mid = (low + up) / 2;
    if (x < a[mid]) {
        up = mid - 1;
    } else {
        if (x > a[mid]) {
            low = mid + 1;
        } else {
            return;
        }
    }
}

for (i = *a1; i > low; i--) {
    a[i] = a[i - 1];
}
a[low] = x;
++*a1;
}
```

```
void InputData(int a[], int *n) {
    int i;
    *n = 0;
    printf("Input the integers (to end input -9999)\n");
    do {
        scanf("%d", &i);
        if (i == -9999) {
            break;
        }
        InsertSort(a, n, i);
    } while (9);
}
```

```
void InsertTree(TNode **r, int x) {
    TNode *p, *q;
    p = malloc(sizeof(TNode));
    p->d = x;
    p->lchild = p->rchild = NULL;
    if (*r == NULL) {
        *r = p;
        return;
    }
}
```

```

    }
    q = *r;
    while (1) {
        if (x < q->d) {
            if (q->lchild) {
                q = q->lchild;

            } else {
                q->lchild = p;
                return;
            }
        } else {
            if (q->rchild) {
                q = q->rchild;
            } else {
                q->rchild = p;
                return;
            }
        }
    }
}

```

```

int count;
void preorder(TNode *r) {
    if (r) {
        printf("%5d", r->d);
        if (++count % 10 == 0) {
            printf("\n");
        }
        preorder(r->lchild);
        preorder(r->rchild);
    }
}

void inorder(TNode *r) {
    if (r) {
        inorder(r->lchild);
        printf("%5d", r->d);
        if (++count % 10 == 0) {
            printf("\n");
        }
        inorder(r->rchild);
    }
}

```

```
}

int height(TNode *r) {
    int h1, h2;
    if (r == NULL) {
        return 0;
    } else {
        h1 = height(r->lchild);
        h2 = height(r->rchild);
        return 1 + (h1 > h2 ? h1 : h2);
    }
}

void CreateAVL(int a[], int s, int e, TNode ** r) {
    int i;
    if (e < s) {
        return;
    }
    i = (s + e) / 2;
    InsertTree(r, a[i]);
    CreateAVL(a, s, i - 1, r);
    CreateAVL(a, i + 1, e, r);
}

int main(void) {
    TNode * r = NULL;
    int a[MAX], n;
    InputData(a, &n);
    if (n) {
        CreateAVL(a, 0, n - 1, &r);
        puts("The preorder is as follows:");
        count = 0;
        preorder(r);
        puts("The inorder is as follows:");
        count = 0;
        inorder(r);
        n = height(r);
        printf("\nThe height of the tree is:%-2d\n", n);
    } else {
        puts("No data are inputed, the tree is a null tree!");
    }
    return EXIT_SUCCESS;
}
```

运行结果 4

```

Input the integers (to end input -9999)
12 433 65 76 33 3 656 87 878 99 -9999
The preorder is as follows:
    76    12    3    33    65    433    87    99    656    878
The inorder is as follows:
    3    12    33    65    76    87    99    433    656    878

The height of the tree is:4

```

图 4-1 求 AVL 树

例题 5：最小生成树

编写一个C程序，它能读入一个带权连通无向图G，生成G的最小生成树T，并输出T的各条边的权之和。

图的输入形式如：

```
n i0 j0 w0 i1 j1 w1 ... ik jk wk -1 -1 -1
```

代码清单 5-1

```

#include <stdio.h>
#include <stdlib.h>

void exit(int);
#define MAX 100
#define M (MAX*(MAX-1)/2)

enum boolean {
    FALSE, TRUE
};

typedef struct edge {
    int i, j, w;
} Edge;

/**
 * 冒泡排序
 */
void bsort(Edge a[], int n) {
    Edge t;
    int i, j, ok;
    for (i = 0; i < n; i++) {

```

```
    ok = TRUE;
    for (j = 0; j < (n - 1); j++) {
        if (a[j].w > a[j + 1].w) {
            t = a[j];
            a[j] = a[j + 1];
            a[j + 1] = t;
            ok = FALSE;
        }

    }
    if (ok) {
        break;
    }
}

/**
 * 读
 */
int read(Edge a[], int n) {
    int i, j, w;
    int c = 0;
    do {

        scanf("%d%d%d", &i, &j, &w);
        if (i == -1 || j == -1 || w == -1) {
            break;
        }

        if (i < 0 || i >= n || j < 0 || j >= n || w <= 0 || i == j) {
            printf(
                "Try again: w must be positive and i, j must be in[0.. %d]\n",
                (n - 1));
            continue;
        }
        if (c >= M) {
            puts("Error: Too many edges!");
            exit(-2);
        }
        a[c].i = i;
        a[c].j = j;
        a[c].w = w;
        (c)++;
    }
```

```

    } while (9);

    return c;
}

int SameTree[MAX];
int InSameTree(int i, int j) {
    int s;
    if (SameTree[i] == -1) {
        if (SameTree[j] == -1) {
            SameTree[i] = j;
            SameTree[j] = i;
        } else {
            SameTree[i] = SameTree[j];
            SameTree[j] = i;
        }
        return FALSE;
    } else {
        if (SameTree[j] == -1) {
            SameTree[j] = SameTree[i];
            SameTree[i] = j;
            return FALSE;
        } else {
            s = i;
            while (SameTree[s] != i) {
                if (SameTree[s] == j) {
                    return TRUE;
                }
                s = SameTree[s];
            }
            SameTree[s] = SameTree[j];
            SameTree[j] = i;
            return FALSE;
        }
    }
}

```

```

void MST(Edge a[], int n, int vnum) {
    int c = 0, i, sum = 0;
    puts("The MST is as follows:");
    for (i = 0; i < n; i++) {
        if (InSameTree(a[i].i, a[i].j)) {

```

```
        continue;
    }
    sum += a[i].w;
    printf("(V%d, V%d) ", a[i].i, a[i].j);
    if (++c % 8 == 0) {
        puts("");
    }
    if (c >= (vnum - 1)) {
        printf("\nThe sum is: %d\n", sum);
        return;
    }
}

if (c < (vnum - 1)) {
    puts("The graph is not connected!");
}
}

int main(void) {
    Edge a[M];
    int i, n, vnum;

    puts("Enter vexnum and edges with weight:");
    scanf("%d", &vnum);
    if (vnum < 1 || vnum > MAX) {
        printf("Error: vexnum must be in [1.. %d]\n", MAX);
        return EXIT_FAILURE;
    }
    n = read(a, vnum);
    bsort(a, n);
    for (i = 0; i < vnum; i++) {
        SameTree[i] = -1;
    }
    MST(a, n, vnum);
    return EXIT_SUCCESS;
}
```

运行结果 5-1

```

Enter vexnum and edges with weight:
6 0 1 1 0 2 33 0 3 43 0 4 5 0 5 32 1 2 3 1 3 4 1 4 5 1 5
76 2 3 2 2 4 5 2 5 6 3 4 5 3 5 665 4 5 4 -1 -1 -1
The MST is as follows:
(V0, V1) (V2, V3) (V1, V2) (V4, V5) (V0, V4)
The sum is: 15

```

图 5-1 求最小生成树——解法 1

代码清单 5-2

```

#include <stdio.h>
#include <stdlib.h>

void exit(int);

#define MAX 100
#define MAXINT 32767

void read(int a[][MAX], int n) {
    int i, j, w;
    do {
        scanf("%d%d%d", &i, &j, &w);
        if (i == -1 || j == -1 || w == -1) {
            break;
        }

        if (i < 0 || i >= n || j < 0 || j >= n || w <= 0 || i == j) {
            printf(
                "Try again: w must be positive and i, j must be in[0.. %d]\n",
                (n - 1));
            continue;
        }
        a[i][j] = w;
    } while (9);
}

void MST(int a[][MAX], int n) {
    int c = 0, i, j, min_i, min_j, min_v, sum = 0;
    puts("The MST is as follows:");
    a[0][0] = -1;
    for (c = 0; c < (n - 1); c++) {

```

```

    min_v = MAXINT;
    min_i = -1;
    min_j = -1;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            if (i != j && ((a[i][i] == -1 && a[j][j] != -1) || (a[i][i]
                != -1 && a[j][j] == -1)) && a[i][j] < min_v) {
                min_v = a[i][j];
                min_i = i;
                min_j = j;
            }
        }
    }
    if (min_i != -1) {
        printf("(V%d, V%d) ", min_i, min_j);
        if ((c + 1) % 8 == 0) {
            puts("");
        }
        sum += min_v;
        a[min_i][min_i] = a[min_j][min_j] = -1;
    } else {
        puts("The graph is not connected!");
        exit(-2);
    }
}
printf("\nThe sum is: %d\n", sum);
}

int main(void) {
    int a[MAX][MAX], i, j;
    int vnum;
    puts("Enter vexnum and edges with weight:");
    scanf("%d", &vnum);
    if (vnum < 1 || vnum > MAX) {
        printf("Error: vnum must be in[1.. %d]\n", MAX);
        return EXIT_FAILURE;
    }
    for (i = 0; i < vnum; i++) {
        for (j = 0; j < vnum; j++) {
            a[i][j] = MAXINT;
        }
    }
}

```

```
read(a, vnum);  
MST(a, vnum);  
return EXIT_SUCCESS;  
}
```

运行结果 5-2

```
Enter vexnum and edges with weight:  
6 0 1 1 0 2 33 0 3 43 0 4 5 0 5 32 1 2 3 1 3 4 1 4  
5 1 5 76 2 3 2 2 4 5 2 5 6 3 4 5 3 5 665 4 5 4 -1 -1 -1  
The MST is as follows:  
(V0, V1) (V1, V2) (V2, V3) (V0, V4) (V4, V5)  
The sum is: 15
```

图 5-2 求最小生成树——解法 2